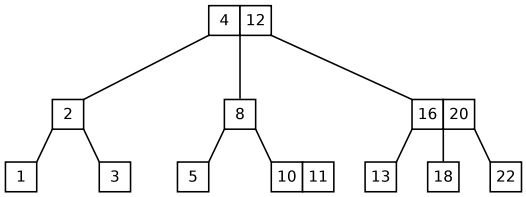
Solutions to Week 10 Lab Sheet conceptual questions

1. For a B-tree with minimum degree $t = 3$, insert the following sequence of keys in order:

   $$\{S, Z, G, Y, B, N, D, E, F, U, I, V, M, X, H\}.$$

   Draw the state of the B-tree after each insertion.

2. The figure below shows the current state of a B-tree with minimum degree $t = 2$. Delete the keys in the following order: $\{1, 22, 16, 8, 18, 5\}$, and draw the state of the tree after each deletion.



Ans: 1-2

See working attached.

3. For a B-tree $B(t)$ with a minimum degree parameter t containing N elements, derive a **lower bound** on the height h of $B(t)$ as a function of t and N.

Ans: 3

In a B-tree of minimum degree t, each node (except possibly the root) contains at least $t - 1$ and at most $2t - 1$ elemnts.

A lower bound on the shallowest possible B-tree containing N elements can be reasoned by imagining a state of the tree where each node is full (with $(2t - 1)$ elements per node).

Below is a table accounting for the number of nodes and number of elements at each level of a B-tree:

Level	**Number of nodes**	**Number of elements**
0	1	$2t - 1$
1	$2t$	$2t(2t - 1)$
2	$(2t)^2$	$(2t)^2(2t - 1)$
3	$(2t)^3$	$(2t)^3(2t - 1)$
$\vdots$	$\vdots$	$\vdots$
h	$(2t)^h$	$(2t)^h(2t - 1)$

This implies

$$\begin{aligned} N &= \sum_{i=0}^{h}(2t)^i(2t-1) \\ &= (2t-1)\sum_{i=0}^{h}(2t)^i \\ &= \cancel{(2t-1)}\frac{(2t)^{h+1}-1}{\cancel{2t-1}} \\ &= (2t)^{h+1}-1 \\ \implies N+1 &= (2t)^{h+1} \\ \log_{2t}(N+1) &= h+1 \\ \implies h &= \log_{2t}(N+1)-1 \\ &= \Omega(\log N) \end{aligned}$$

(Note: We used a Big-Omega notation above because this is reasoning a lower bound. In general, a function f is $\Omega(g)$ if and only if for some positive constants k and m $f(x) \geq kg(x) \geq 0\ \forall n \geq m$. In plain words, this means f grows at least as quickly as g. This is same as saying $g(x) \leq \frac{f(x)}{k} \forall n \geq m$ which means that $g = O(f)$.)

4. For a B-tree $B(t)$ with a minimum degree parameter t containing N elements, derive an **upper bound** on the height h of $B(t)$ as a function of t and N.

Ans: 4

In a B-tree of minimum degree t, each node (except possibly the root) contains at least $t-1$ and at most $2t-1$ elements.

An upper bound on the height of the tallest possible B-tree containing N elements can be reasoned by imagining a state of the tree where each node has the minimum allowable number of elements. This means, barring the root (which may contain as few as 1 element when $N > 0$), every other node contains exactly $(t-1)$ elements.

Below is a table accounting for the number of nodes and number of elements at each level of a B-tree:

Level	Number of nodes	Number of elements
0	1	1
1	2	$2(t-1)$
2	$2t$	$2t(t-1)$
3	$2t^2$	$2t^2(t-1)$
4	$2t^3$	$2t^3(t-1)$
$\vdots$	$\vdots$	$\vdots$
h	$2t^{h-1}$	$2t^{h-1}(t-1)$

This implies

$$\begin{aligned} N &= 1+\sum_{i=0}^{h-1}2t^i(t-1) \\ &= 1+2(t-1)\sum_{i=0}^{h-1}t^i \\ &= 1+2\ \cancel{(t-1)}\frac{t^h-1}{\cancel{t-1}} \\ &= 2t^h-1 \\ \implies N+1 &= 2t^h \\ \implies \frac{N+1}{2} &= t^h \\ \implies h &= \log_t\left(\frac{N+1}{2}\right) \\ &= O(\log N). \end{aligned}$$

5. Can a B-tree $B(t)$ with $N \geq 2t$ elements ever reach a state in which **all** nodes are fully occupied (or, conversely, **all** nodes at minimum possible occupancy) through a sequence of insert and delete operations? Justify your answer.

Ans: 5

No. Although we used these extreme states for reasoning about upper and lower bounds in questions 3 and 4, they are not realizable through arbitrary sequences of insertions and deletions in general B-trees.

This is because insertion in a B-tree requires splitting nodes that that are full during traversal preventing the entire tree from remaining uniformly full. Similarly, deletion requires merging during traversal when two siblings nodes along the path of traversal are sitting at bare minimum number of elements, preventing getting to a state where all nodes are uniformly at minimum capacity.

(Having said this, note that it is not uncommon in practice, when B-tree indexes are meant to be static (meaning insertions/deletions are unnecessary, and one uses just for searching), to preload such a tree in a state where each node is filled to its maximum capacity for performance gains.)

6. Suppose that we insert the keys $\{1, 2, \ldots, n\}$ in order into an initially empty B-tree with minimum degree $t = 2$. Provide upper and lower bounds on the number of nodes in the resulting B-tree.

Ans: 6

First, the B-tree parameter $t = 2$ implies that each node can contain a minimum of 1 key and a maximum of 3 keys.

Clearly for $n \leq 2t - 1 = 3$, the total number of nodes is exactly 1 (root node). The problem is interesting when inserting $n \geq 2t = 4$ keys in strictly increasing order. Since all insertions in a B-tree are performed in a leaf node, and each new key is strictly larger than all existing keys in the tree, every insertion in this scenario must occur in the rightmost leaf of the current B-tree.

Thus, each insertion operation involves traversing the rightmost path from the root to the rightmost leaf.* Whenever a node on this path becomes full (containing 3 keys for $t = 2$), it gets split into two nodes with one key each, with the middle key promoted to the parent node. Most importantly, for this scenario of strictly increasing key insertions, when $n \geq 4$, nodes not on the rightmost path *always contain exactly 1 key.*

From the previous analysis answering question 4, we saw that the maximum height of the B-tree when $t = 2$ is

$$h_{\max} = \log_2 \left(\frac{n+1}{2} \right),$$

which occurs when *all* nodes in the tree contains exactly $t - 1 = 1$ key. From this, an upper bound on the total number of nodes is clearly $n_{\max} \leq n$. (In fact, it can be tightened to $n - 1$ since the rightmost leaf node has ≥ 2 keys for all $n \geq 4$.)

For the lower bound, note that there are $h_{\max} + 1$ nodes along the rightmost path (including both the root and the rightmost leaf). This implies that there are at least $N - (h_{\max} + 1)$ singleton nodes (containing exactly one key) not on this path. Conservatively counting only the root node along the rightmost path gives a lower bound on the total number of nodes:

$$n_{\min} \geq N - h_{\max}.$$

7. What is the worst-case space complexity of a B-tree $B(t)$ storing N elements?

Ans: 7

$O(N)$.

This is straightforward. Each of the N elements is stored in exactly one B-tree node. Each node contains at most $2t - 1$ elements and $2t$ child pointers, requiring space per node that is linear in t (which is typically a fixed constant aligned to the `PAGE_SIZE` parameter on some computer). The total number of nodes in the B-tree is bounded by $O(N)$, regardless of t. Therefore, the worst-case space complexity is $O(N)$.

*Consider performing a dozen or so insertions on paper to see why this always occurs. This will give you practice for B-tree insertion.

8. Analyze the worst-case time complexities of the search, insertion, and deletion operations, respectively, in a B-tree $B(t)$.

Ans: 8

All these operations derive their worst-case time-bounds using the upper bound on tree height (derived in question 4) which is $h = O(\log N)$. Specifically:

- **Search:** A search starts from the root of the B-tree and, in the worst case, proceeds down to a leaf. At each node, the search examines at most $2t-1$ keys, using binary search to locate the appropriate key-interval and then follows the corresponding subtree (see pseudocode below in the answer to question 9).

  Binary search within a node takes $O(\log t)$ time, and since t is a fixed constant, this can be treated as $O(1)$ time.

  The search visits at most one node per level, so the total worst-case time is proportional to the height of the tree: $O(h \log t) = O(h) = O(\log N)$.

- **Insertion:** To insert a key, the B-tree is always traversed from the root to the appropriate leaf. At each node along the path, a search among the node's keys determines which child to descend into. Since each node contains at most $2t-1$ keys, this search can be performed using binary search in $O(\log t)$ time.

  Whenever a full node (containing $2t-1$ keys) is encountered, it must be split. Splitting a node involves promoting the middle key to the parent and dividing the remaining keys into two new nodes, which requires $O(t)$ time per split.

  In the worst case, a split may occur at every level from the root down to the leaf. The worst-case height of the tree is $h = O(\log N)$. Therefore, the total worst-case time complexity of insertion is $O\big(h(t + \log t)\big) = O(h) = O(\log N)$ $(\because t$ is a constant.)

- **Deletion:** Deletion in a B-tree ultimately is performed at a leaf node (case 1 deletion). Even when deleting a key from an internal node (case 2 deletion), the problem reduces to removing a key from a leaf (case 1 deletion). Therefore, deletion always requires a traversal from the root to the leaf containing the key to be deleted.

  During this traversal, if a non-root node has the minimum number of keys $(t-1)$, applying case 3, either

  (a) a key is rotated from a sibling node or

  (b) two sibling nodes get merged.

  Each such case 3 operation takes $O(t)$ time in addition to the binary search that takes $O(\log t$ effort at each node. Since the worst case height is $O(\log N)$, the worst-case time complexity of deletion becomes $O\big(h(t + \log t)\big) = O(h) = O(\log N)$ $(\because t$ is a constant.)

9. Write pseudocode for a function `search`(P, x) in a B-tree, where P is any node of a B-tree and x is the key being searched for in the (sub)B-tree rooted at P. The function should first *binary search* the keys stored in node P. If $x \notin P$, the function recursively searches an appropriate subtree of P where it exits. The return value is true/false indicating whether or not x was found in the subtree rooted at P. (A driver function is assumed to initiate the search with P as the root node of the full B-tree.)

   Justify that your search procedure terminates correctly.

   Ans: 9

```
1  search(P,x)
2  /* * Assumptions:
3   *   n_k = number of keys in node P
4   *   k[1...n_k] is the array storing the keys in node P
5   *     such that  -∞ < k[1] < k[2] < ··· < k[n_k] <  +∞
                      =k[0]                          =k[n_k+1]
6   *   T[0...n_k] is the array subtrees stored in P
7   *     such that {T[0]} < k[1] < {T[1]} < k[2] < {T[2]} < ··· < k[n_k] < {T[n_k]} */
8
9    lo ← 0
10   hi ← n_k + 1
11
12   while (lo < hi − 1)
13   /* maintain loop invariant k[lo] ≤ x<k[hi] */
14     mid ← ⌊(lo+hi)/2⌋;
15     if (x ≥ k[mid]) lo ← mid
16     else            hi ← mid
17
18   /* At termination  k[lo] ≤ x < k[lo + 1]. So... */
19   if (lo ≥ 1 and k[lo] = x) return true
20   else if (P is leaf)    return false
21   return search(T[lo],x)
```

Justification of termination: The loop condition is lo < hi − 1. $\implies$ lo + 1 < hi $\implies$ lo + 2 ≤ hi. Thus, the while loop runs as long as [lo, hi) contains at least two indices.
So, inside the loop it follows that lo < mid < hi. Further,

- if lo ← mid, then lo strictly increases, and
- if hi ← mid, then hi strictly decreases.

Therefore, in every iteration, the interval size hi − lo strictly decreases.
Since hi − lo is always a positive integer, it cannot decrease indefinitely and the loop terminates when hi − lo = 1, given the loop condition lo < hi − 1.
The recursive procedure also terminates. If the key is found in the current node P, the procedure immediately returns true. Otherwise, if P is a leaf node, the procedure returns false.
If neither condition holds, the procedure recursively searches exactly one child subtree of P. Since every recursive call descends one level deeper in the B-tree, and a B-tree has finite height, the recursion must eventually reach either:

- a node containing x, or
- a leaf node where x is not present.

Hence, the recursive search procedure terminates.
Justification of terminating correctly: The loop invariant maintained throughout the binary search is k[lo] ≤ x < k[hi].
Initially, this holds since $k[0] = -\infty$ and $k[n_k + 1] = +\infty$.
At each iteration:

- if $x \geq k$[mid], then setting lo ← mid preserves the invariant since k[mid] ≤ x;
- otherwise, setting hi ← mid preserves the invariant since $x < k$[mid].

When the loop terminates, hi − lo = 1, so the invariant implies
k[lo] ≤ x < k[lo + 1].
Hence:

- if k[lo] = x, then the key has been found and the algorithm correctly returns true;
- otherwise, k[lo] < x < k[lo + 1], so by the B-tree ordering property, if x exists in the subtree rooted at P, it must lie in subtree T[lo].

Therefore, each recursive call searches the unique subtree that could possibly contain x, and the algorithm correctly returns whether or not x occurs in the subtree rooted at P.

-=o0o=-
END
-=o0o=-